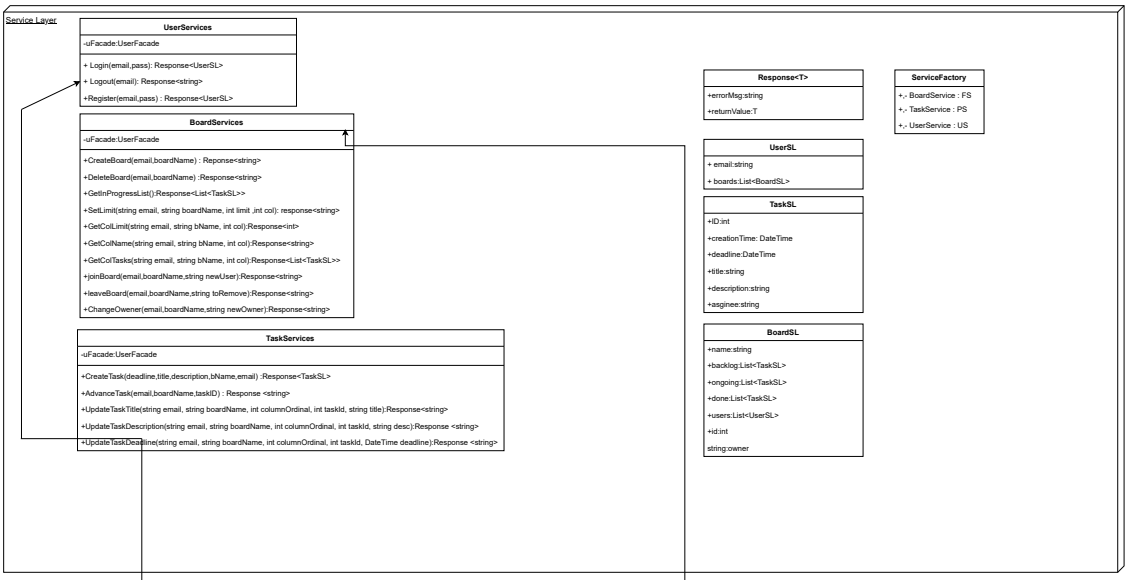
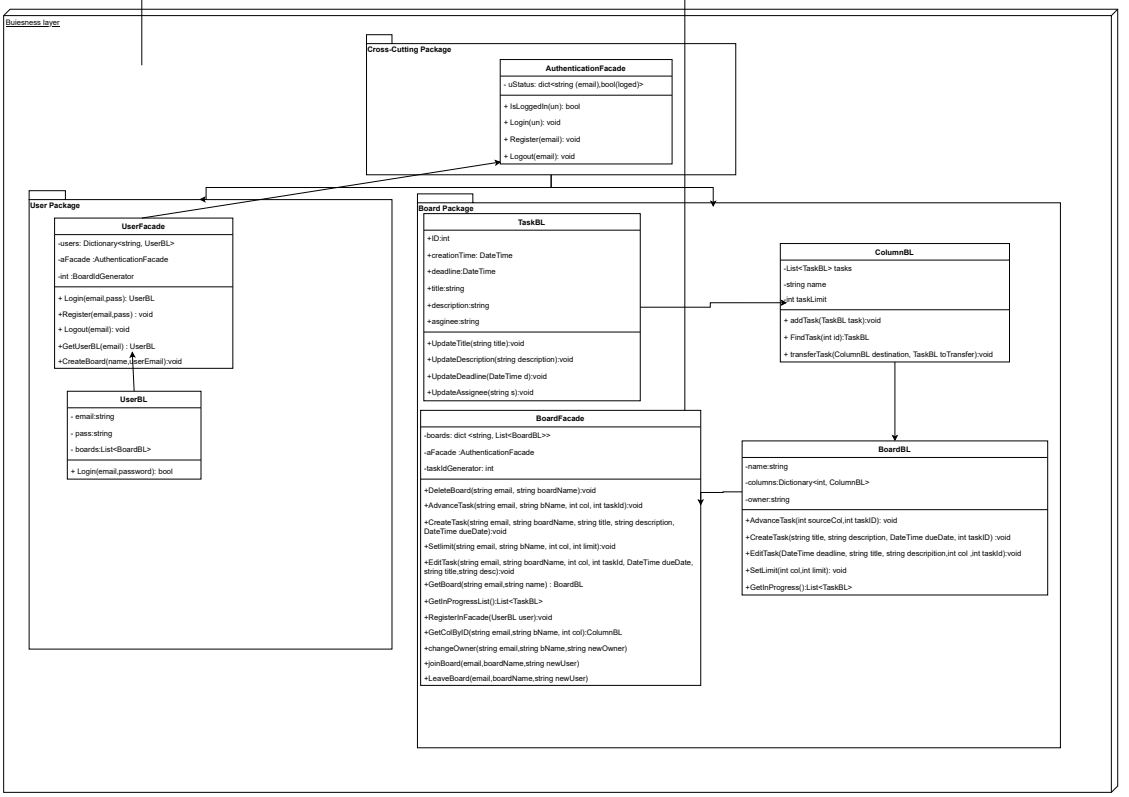
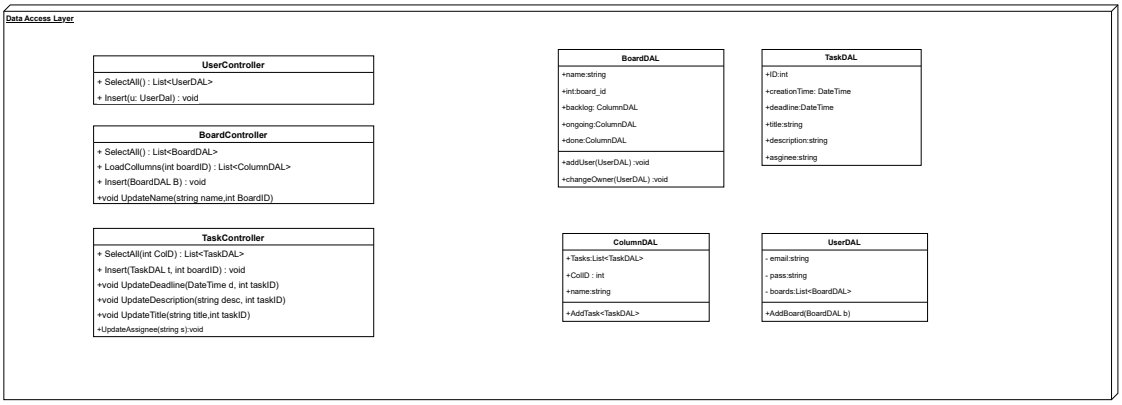
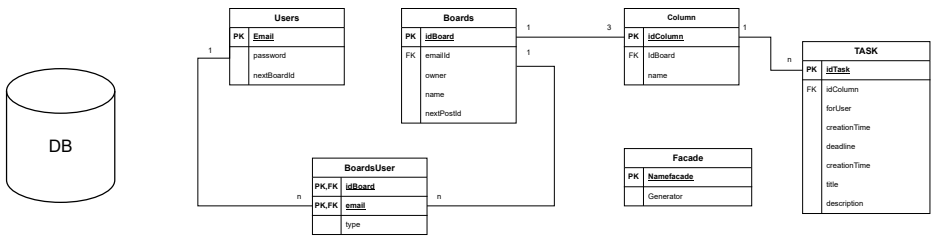

Service Layer
UserServices
-uFacade:UserFacade
+ Login(email,pass): Response<UserSL>
+ Logout(email): Response<string>
+Register(email,pass) : Response<UserSL>
BoardServices
-uFacade:UserFacade
+CreateBoard(email,boardName) : Reponse<string>
+DeleteBoard(email,boardName) :Response<string>
+GetInProgressList():Response<List<TaskSL>>
+SetLimit(string email, string boardName, int limit ,int col): response<string>
+GetColLimit(string email, string bName, int col):Response<int>
+GetColName(string email, string bName, int col):Response<string>
+GetColTasks(string email, string bName, int col):Response<List<TaskSL>>
+joinBoard(email,boardName,string newUser):Response<string>
+leaveBoard(email,boardName,string toRemove):Response<string>
+ChangeOwener(email,boardName,string newOwner):Response<string>
TaskServices
-uFacade:UserFacade
+CreateTask(deadline,title,description,bName,email) :Response<TaskSL>
+AdvanceTask(email,boardName,taskID) : Response <string>
+UpdateTaskTitle(string email, string boardName, int columnOrdinal, int taskId, string title):Response<string>
+UpdateTaskDescription(string email, string boardName, int columnOrdinal, int taskId, string desc):Response <string>
+UpdateTaskDeadline(string email, string boardName, int columnOrdinal, int taskId, DateTime deadline):Response <string>
Response<T>
+errorMsg:string
+returnValue:T
ServiceFactory
+,- BoardService : FS
+,- TaskService : PS
+,- UserService : US
UserSL
+ email:string
+ boards:List<BoardSL>
TaskSL
+ID:int
+creationTime: DateTime
+deadline:DateTime
+title:string
+description:string
+asginee:string
BoardSL
+name:string
+backlog:List<TaskSL>
+ongoing:List<TaskSL>
+done:List<TaskSL>
+users:List<UserSL>
+id:int
string:owner


Buiesness layer
Cross-Cutting Package
AuthenticationFacade
- uStatus: dict<string (email),bool(loged)>
+ IsLoggedIn(un): bool
+ Login(un): void
+ Register(email): void
+ Logout(email): void
User Package
UserFacade
-users: Dictionary<string, UserBL>
-aFacade :AuthenticationFacade
-int :BoardIdGenerator
+ Login(email,pass): UserBL
+Register(email,pass) : void
+ Logout(email): void
+GetUserBL(email) : UserBL
+CreateBoard(name,UserEmail):void
UserBL
- email:string
- pass:string
- boards:List<BoardBL>
+ Login(email,password): bool
Board Package
TaskBL
+ID:int
+creationTime: DateTime
+deadline:DateTime
+title:string
+description:string
+asginee:string
+UpdateTitle(string title):void
+UpdateDescription(string description):void
+UpdateDeadline(DateTime d):void
+UpdateAssignee(string s):void
ColumnBL
-List<TaskBL> tasks
-string name
-int taskLimit
+ addTask(TaskBL task):void
+ FindTask(int id):TaskBL
+ transferTask(ColumnBL destination, TaskBL toTransfer):void
BoardFacade
-boards: dict <string, List<BoardBL>>
-aFacade :AuthenticationFacade
-taskIdGenerator: int
+DeleteBoard(string email, string boardName):void
+AdvanceTask(string email, string bName, int col, int taskId):void
+CreateTask(string email, string boardName, string title, string description, DateTime dueDate):void
+SetLimit(string email, string bName, int col, int limit):void
+EditTask(string email, string boardName, int col, int taskId, DateTime dueDate, string title,string desc):void
+GetBoard(string email,string name) : BoardBL
+GetInProgressList():List<TaskBL>
+RegisterInFacade(UserBL user):void
+GetColByID(string email,string bName, int col):ColumnBL
+changeOwner(string email,string bName,string newOwner)
+joinBoard(email,boardName,string newUser)
+LeaveBoard(email,boardName,string newUser)
BoardBL
-name:string
-columns:Dictionary<int, ColumnBL>
-owner:string
+AdvanceTask(int sourceCol,int taskID): void
+CreateTask(string title, string description, DateTime dueDate, int taskID) :void
+EditTask(DateTime deadline, string title, string description,int col ,int taskId):void
+SetLimit(int col,int limit): void
+GetInProgress():List<TaskBL>


Data Access Layer
UserController
+ SelectAll() : List<UserDAL>
+ Insert(u: UserDal) : void
BoardController
+ SelectAll() : List<BoardDAL>
+ LoadCollumns(int boardID) : List<ColumnDAL>
+ Insert(BoardDAL B) : void
+void UpdateName(string name,int BoardID)
TaskController
+ SelectAll(int ColD) : List<TaskDAL>
+ Insert(TaskDAL t, int boardID) : void
+void UpdateDeadline(DateTime d, int taskID)
+void UpdateDescription(string desc, int taskID)
+void UpdateTitle(string title,int taskID)
+UpdateAssignee(string s):void
BoardDAL
+name:string
+int:board_id
+backlog: ColumnDAL
+ongoing:ColumnDAL
+done:ColumnDAL
+addUser(UserDAL) :void
+changeOwner(UserDAL) :void
TaskDAL
+ID:int
+creationTime: DateTime
+deadline:DateTime
+title:string
+description:string
+asginee:string
ColumnDAL
+Tasks:List<TaskDAL>
+ColID : int
+name:string
+AddTask<TaskDAL>
UserDAL
- email:string
- pass:string
- boards:List<BoardDAL>
+AddBoard(BoardDAL b)


DB
Users
PK
Email
password
nextBoardId
Boards
PK
idBoard
FK
emailId
owner
name
nextPostId
Column
PK
idColumn
FK
IdBoard
name
TASK
PK
idTask
FK
idColumn
forUser
creationTime
deadline
creationTime
title
description
BoardsUser
PK,FK
idBoard
PK,FK
email
type
Facade
PK
Namefacade
Generator
1
3
1
1
n
1
n
n